

5 Object Oriented Programing (oop)

Although we write the programs in a general way, the objects themselves cause the behavior to be specific. This lets us write programs that will work correctly even for objects that haven't been invented yet.

5.1 Inheritance

We have already been using the terms Superclass and Subclass. A Superclass is the parent class while a Subclass is the child class. A Subclass inherits the behaviour and variables of the Superclass. A Subclass inherits all the methods and all the encapsulated data variables of its Direct Superclass.

Therefore in the following code JOptionPane inherits all the methods and all the encapsulated data variables that lie within JComponent.

```

+--java.awt.Component
    |
    +--java.awt.Container
        |
        +--javax.swing.JComponent
            |
            +--javax.swing.JOptionPane
```

Class JOptionPane inherits from class JComponent.

But wait a second. Isn't JComponent the Subclass of Container? Correct. So, that tells us JComponent inherits all the methods and all the encapsulated data variables of Container. Container is the Superclass. So, when JOptionPane inherits everything from JComponent (which itself inherited everything from Container), that must mean JOptionPane inherits everything from both JComponent and Container. So, let's put this together: the JOptionPane inherits all the methods and all the encapsulated data

variables in all the Superclasses above it.

To understand how this happens consider this question - When we want to create a copy of some object, what do we do? We use the new keyword to fire off the Subclass object's Constructor and we instantiate the object. And so, if we

javaw.swing

Class JOptionPane

```

java.lang.Object
|
+--java.awt.Component
    |
    +--java.awt.Container
        |
        +--javax.swing.JComponent
            |
            +--javax.swing.JOptionPane
```

61.png - JComponent is the Superclass, JOptionPane is the Subclass